FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Clase 8: Principios fundamentales de la POO

Docentes:

- Ing. Sebastián Balestrini
- Ing. Gustavo Distéfano

Objetivos

- Comprender los principios de encapsulamiento, herencia y polimorfismo.
- Analizar cómo estos principios ayudan a organizar y reutilizar código.
- Aprender a representar relaciones entre clases mediante un diagrama UML básico.
- Ver ejemplos básicos de estos conceptos en C++ y Java.

Herramientas

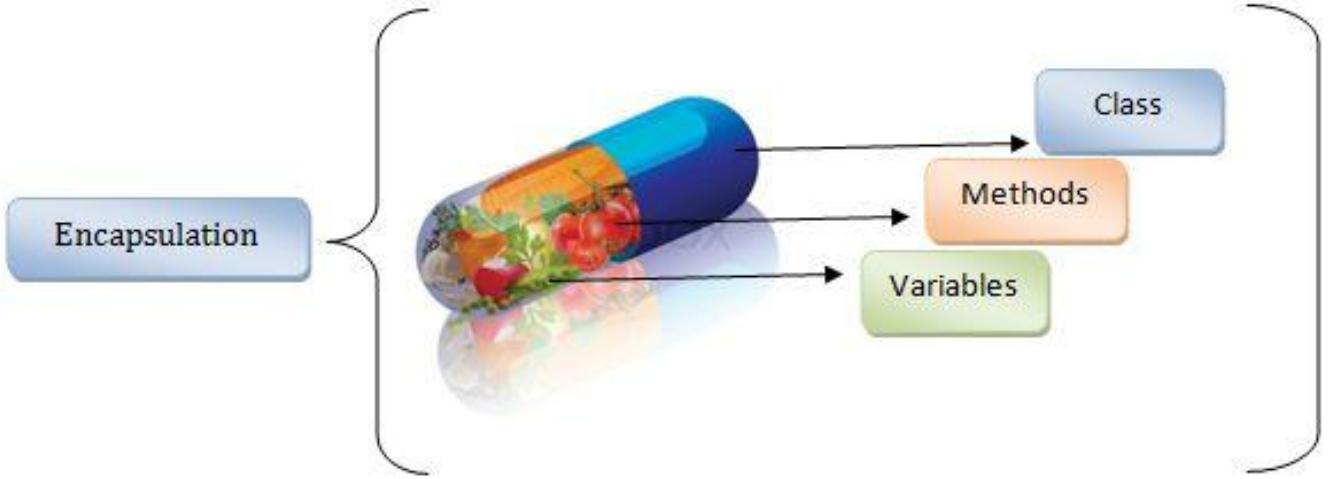
- Lucidchart: https://www.lucidchart.com
- Online GDB: https://www.onlinegdb.com/

Principios de la Programación Orientada a Objetos

“La POO organiza mejor el código mediante principios clave que facilitan reutilización, mantenimiento y legibilidad.”

Encapsulamiento

- Ocultar los detalles internos de una clase y exponer solo lo necesario.
- Protege los atributos usando modificadores de acceso.
- Permite mantener control sobre los datos.



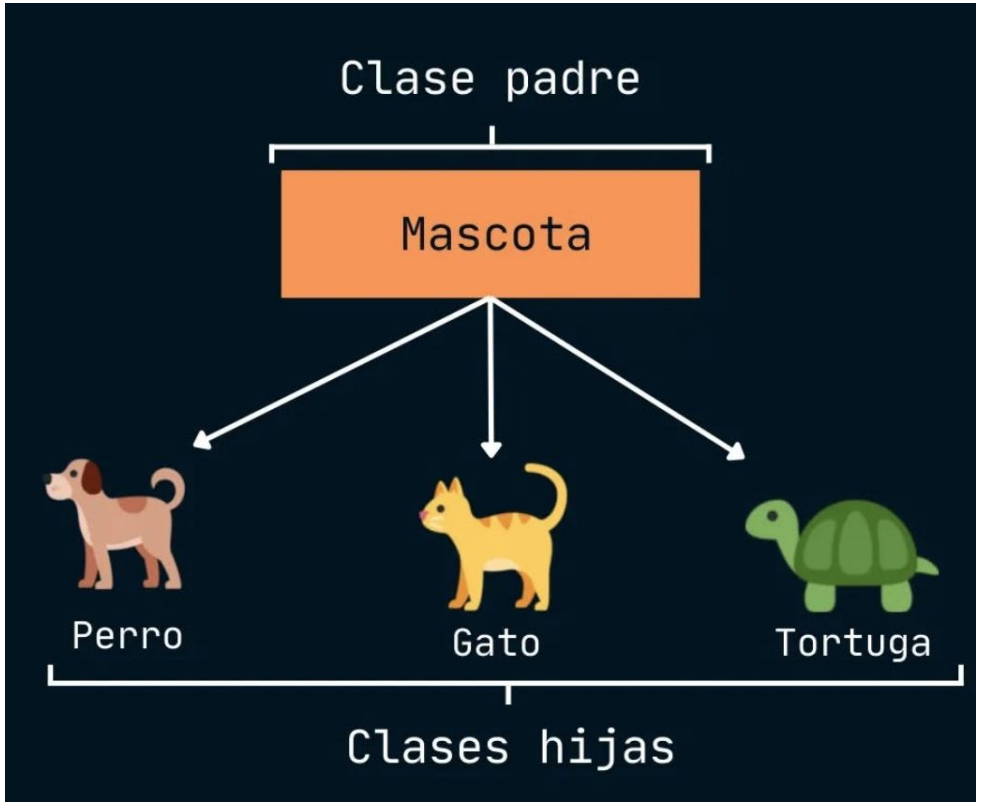
FADENA
FACULTAD DE DEFENSA NACIONAL

Principios de la Programación Orientada a Objetos

Herencia

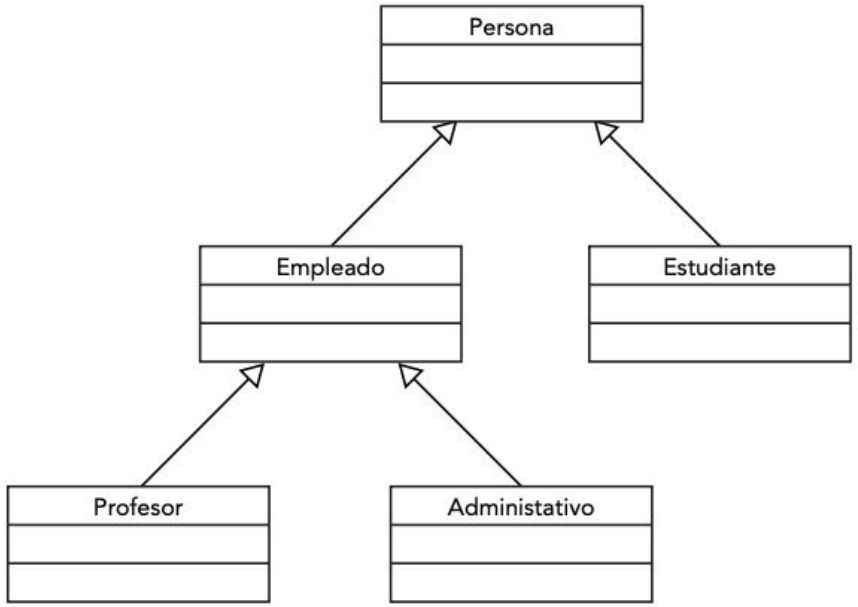
Al implementar una clase y para reutilizar el código una clase puede extender de otra, heredando el comportamiento de la clase extendida. La relación de herencia entre las clases es una relación de «es-un».

- Una clase hija hereda atributos y métodos de una clase padre.
- Permite reutilizar código y extender comportamientos.



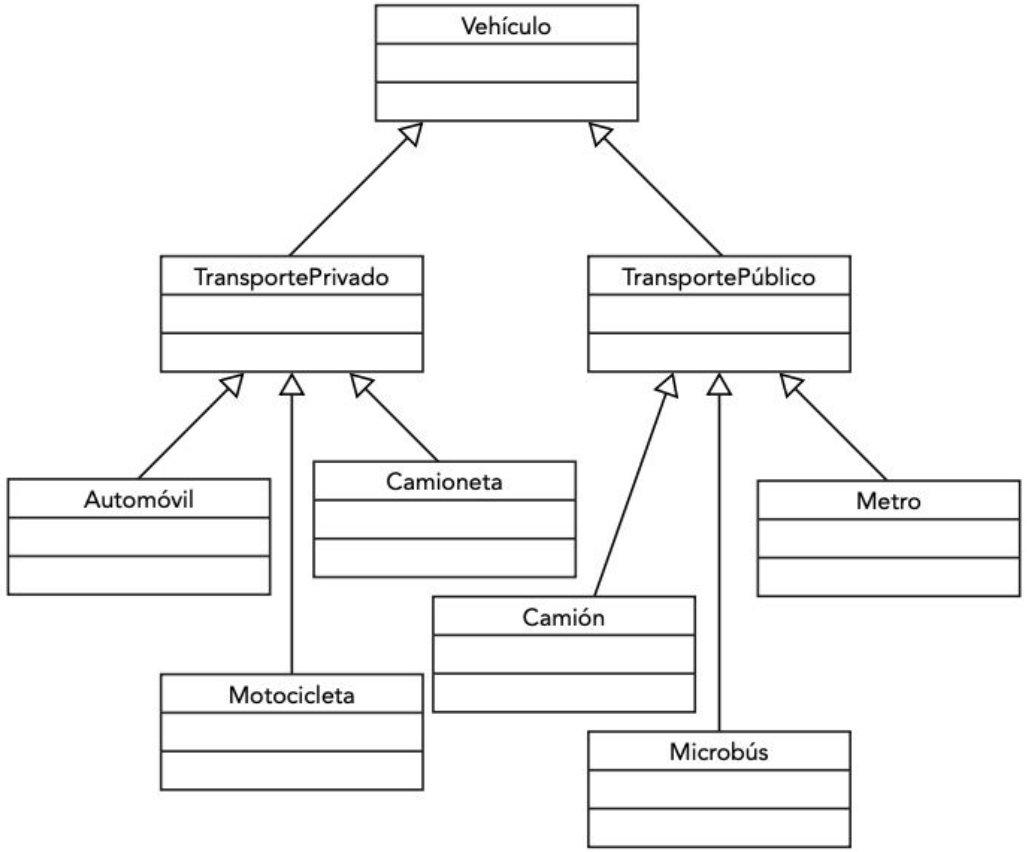
FADENA
FACULTAD DE DEFENSA NACIONAL

Herencia



FADENA
FACULTAD DE DEFENSA NACIONAL

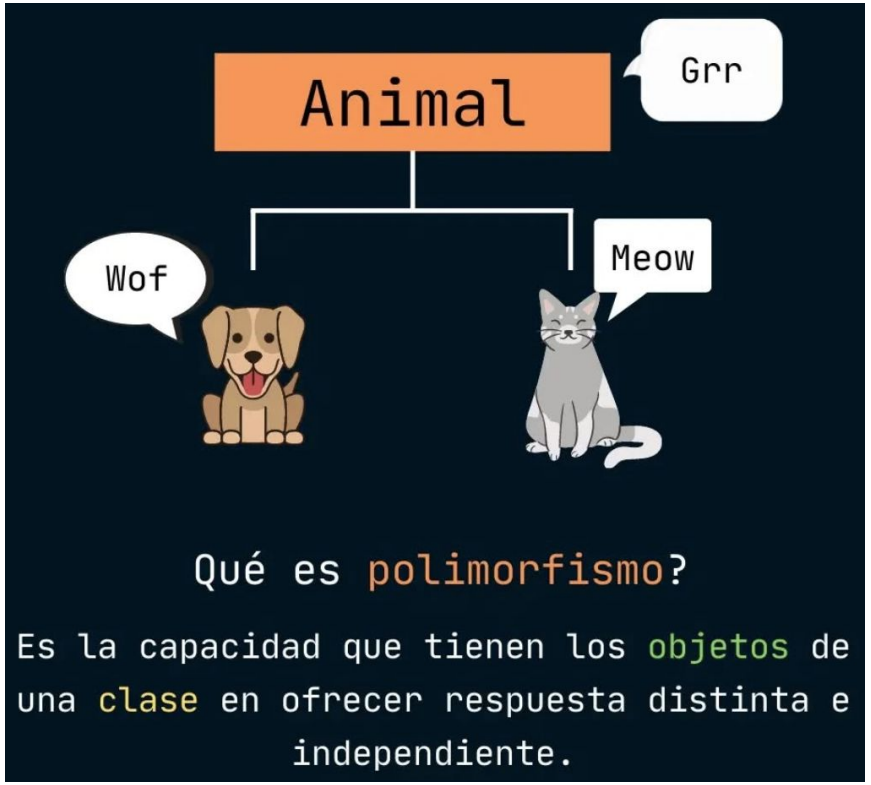
Herencia


Vehículo
TransportePrivado
TransportePúblico
Automóvil
Camioneta
Metro
Camión
Motocicleta
Microbús

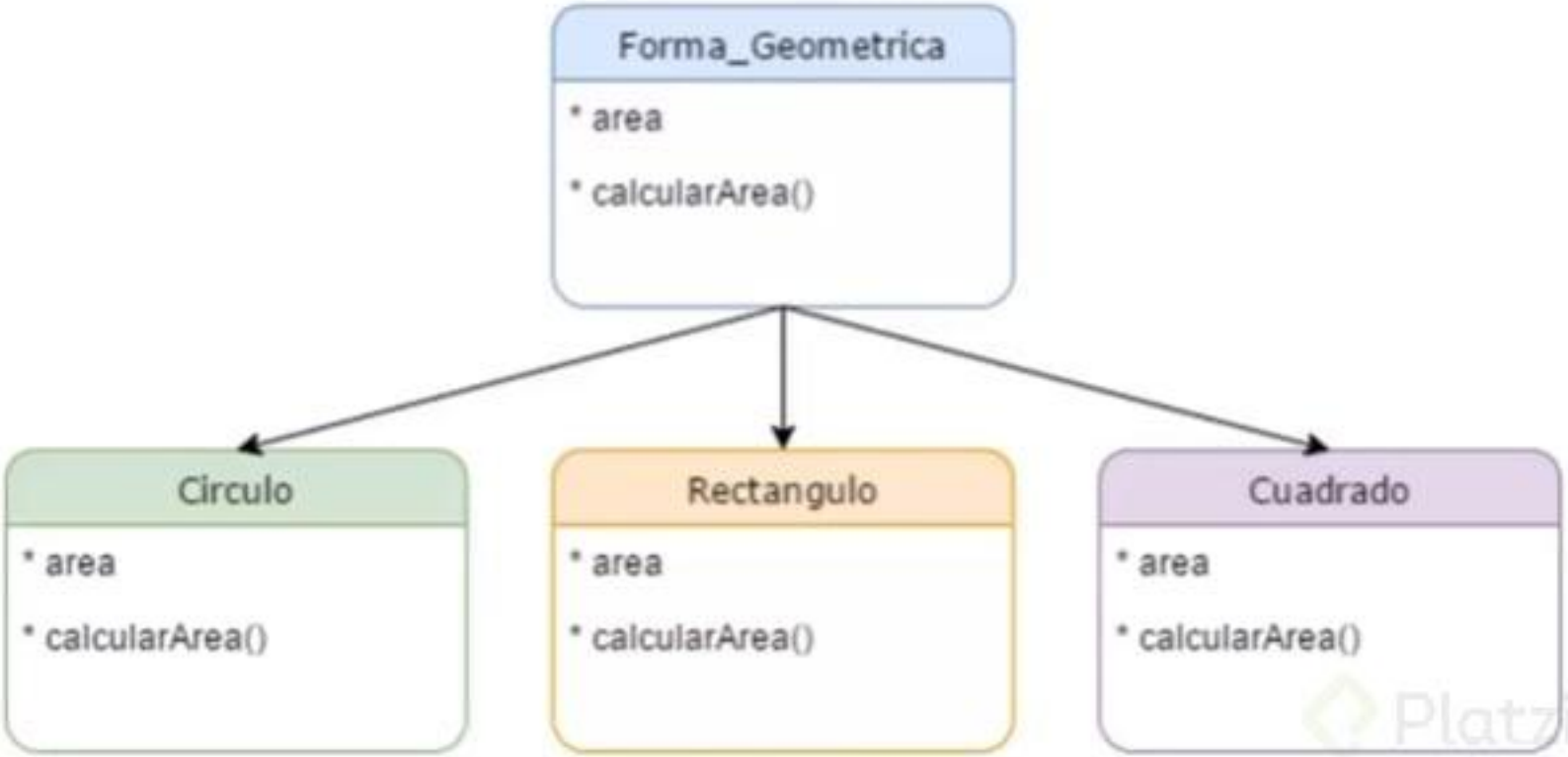
Principios de la Programación Orientada a Objetos

Polimorfismo

- Un mismo método se comporta diferente según el objeto que lo use.
- Permite sobrescribir métodos en clases hijas.



FADENA
FACULTAD DE DEFENSA NACIONAL



FADENA
Platzi

Ventajas de la POO

- Reutilización de atributos y métodos.
- Código mas organizado y fácil de mantener.
- Abstracción.
- Modularidad
- Permite crear sistemas complejos
- Agiliza el desarrollo de software

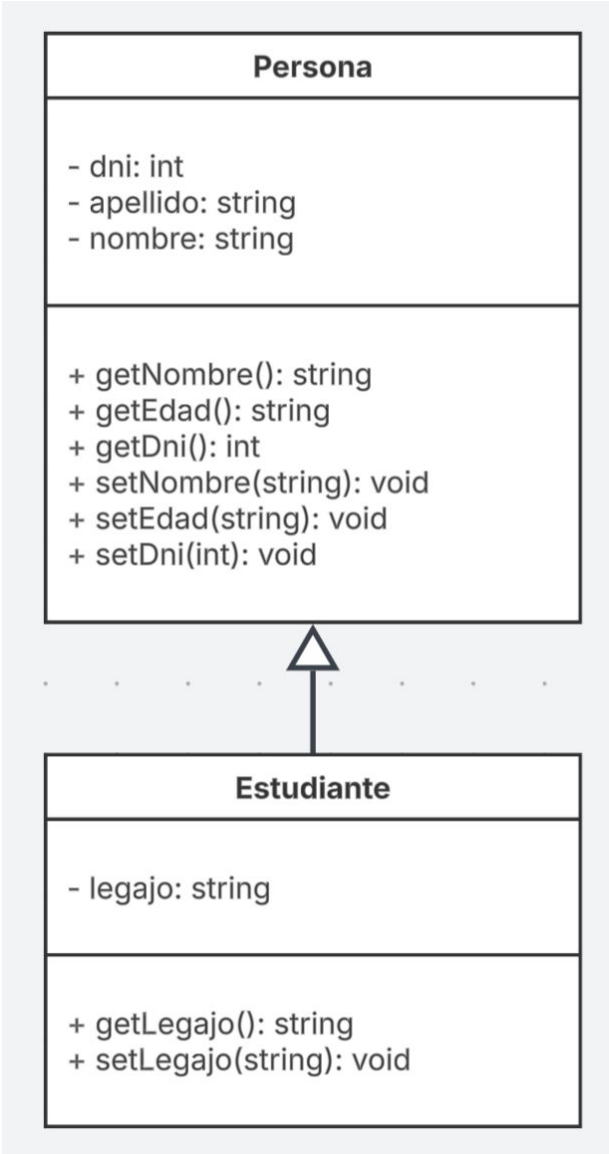
Ventajas de la POO

- Reutilización de atributos y métodos.
- Abstracción: implica simplificar la representación de un objeto centrándose en sus aspectos más relevantes.
- Modularidad
- Mayor Flexibilidad y Escalabilidad
- Permite crear sistemas complejos
- Agiliza el desarrollo de software

UML



FADENA
FACULTAD DE DEFENSA NACIONAL



FADENA
FACULTAD DE DEFENSA NACIONAL

Vayamos al código



FADENA
FACULTAD DE DEFENSA NACIONAL

Encapsulamiento

“Encapsular = ocultar atributos y controlar el acceso mediante métodos (get/set).”

C++

```cpp
#include <iostream>
using namespace std;

class Persona {
private:
    string nombre; // oculto
public:
    void setNombre(string n) { nombre = n; }
    string getNombre() { return nombre; }
};

int main() {
    Persona p;
    p.setNombre("Ana");
    cout << "Nombre: " << p.getNombre() << endl;
}
```

Java

```java
class Persona {
    private String nombre; // oculto
    public void setNombre(String n) { nombre = n; }
    public String getNombre() { return nombre; }
}

class Main {
    public static void main(String[] args) {
        Persona p = new Persona();
        p.setNombre("Seba");
        System.out.println("Nombre: " + p.getNombre());
    }
}
```

Herencia

“Una clase hija hereda atributos y métodos de una clase padre.”

C++

```cpp
#include <iostream>
using namespace std;

class Persona {
public:
    string nombre;
    void saludo() { cout << "Hola, soy " << nombre << endl; }
};

class Estudiante : public Persona {
public:
    int legajo;
};

int main() {
    Estudiante e;
    e.nombre = "Juan";
    e.legajo = 123;
    e.saludo(); // hereda de Persona
}
```

Java

```java
class Persona {
    String nombre;
    void saludo() { System.out.println("Hola, soy " + nombre); }
}

class Estudiante extends Persona {
    int legajo;
}

class Main {
    public static void main(String[] args) {
        Estudiante e = new Estudiante();
        e.nombre = "Juan";
        e.legajo = 123;
        e.saludo(); // hereda de Persona
    }
}
```

Polimorfismo

“Un mismo método se comporta diferente según el objeto

C++

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sonido() { cout << "Un animal hace un sonido\n"; }
};

class Perro : public Animal {
public:
    void sonido() override { cout << "El perro dice: Guau!\n"; }
};

int main() {
    Animal* a = new Perro(); // polimorfismo
    a->sonido(); // llama versión de Perro
}
```

Java

```java
class Animal {
    void sonido() { System.out.println("Un animal hace un sonido"); }
}

class Perro extends Animal {
    void sonido() { System.out.println("El perro dice: Guau!"); }
}

class Main {
    public static void main(String[] args) {
        Animal a = new Perro(); // polimorfismo
        a.sonido(); // llama versión de Perro
    }
}
```

Con virtual, el método se **resuelve dinámicamente** en tiempo de ejecución. En java se hace automáticamente.

FADENA
FACULTAD DE DEFENSA NACIONAL

RESUMEN

Encapsulamiento

- Oculta atributos y expone solo métodos de acceso
- Mejora la organización y seguridad del código

Herencia

- Una clase hija reutiliza atributos y métodos de una clase padre
- Evita duplicar código y permite extender funcionalidades

Polimorfismo

- Un mismo método se comporta diferente según el objeto

UML

- Visualiza clases, atributos, métodos y relaciones
- Flecha con triángulo vacío = herencia

¿ Dudas ?



FADENA
FACULTAD DE DEFENSA NACIONAL

FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Clase 8: Principios fundamentales de la POO

Docentes:

- Ing. Sebastián Balestrini
- Ing. Gustavo Distéfano